Day 03

Object-Oriented Programming

Internship Task Documentation

Topic	Object-Oriented Programming(OOP)
Prepared by	Sakshi Ramesh Karale
Date	16 October 2026

Task 1: Create Student and Employee Classes

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no

    def display(self):
        print("Student Name:", self.name)
        print("Roll Number:", self.roll_no)



class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print("Employee Name:", self.name)
        print("Salary:", self.salary)



s = Student("Sakshi", 101)
e = Employee("Amit", 30000)

s.display()
e.display()
```

Output:

```
Student Name: Sakshi
```

Roll Number: 101

Employee Name: Amit

Salary: 30000

Task 2: Add Properties and Methods

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def calculate_result(self):
        if self.marks >= 35:
            print("Result: Pass")
        else:
            print("Result: Fail")


s = Student("Sakshi", 85)
print("Name:", s.name)
print("Marks:", s.marks)
s.calculate_result()
```

Output:

Name: Sakshi

Marks: 85

Result: Pass

Task 3: Demonstrate Constructor, Inheritance and Encapsulation

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    def display(self):
        print("Name:", self.name)
        print("Salary:", self.__salary)



class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department

    def show department(self):
        print("Department:", self.department)



m = Manager("Amit", 50000, "IT")
m.display()
m.show_department()
```

Output:

Name: Amit

Salary: 50000

Department: IT

Task 4: Student Management Console Application

```
class Student:
    def __init__(self, roll_no, name, marks):
        self.roll_no = roll_no
        self.name = name
        self.marks = marks

    def display(self):
        print("Roll No:", self.roll_no)
        print("Name:", self.name)
        print("Marks:", self.marks)



students = []

while True:
    print("\n1. Add Student")
    print("2. Display Students")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        roll_no = int(input("Enter roll number: "))
        name = input("Enter name: ")
        marks = float(input("Enter marks: "))

        s = Student(roll_no, name, marks)
```

```
        students.append(s)
        print("Student added successfully!")

    elif choice == "2":
        if len(students) == 0:
            print("No students found")
        else:
            for s in students:
                s.display()

    elif choice == "3":
        print("Thank you!")
        break

    else:
        print("Invalid choice")
```

Sample Expected Output:

1. Add Student

2. Display Students

3. Exit

Enter choice: 1

Enter roll number: 101

Enter name: Sakshi

Enter marks: 85

Student added successfully!